Charm Alpha Vaults V2 + 4626fun Master Q&A

Merged source references:

- `docs/charm-alpha-vaults-v2-4626fun-adversarial-audit.md`
- `docs/charm-alpha-vaults-v2-4626fun-executive-brief.md`

Format rule used here: each numbered question is placed directly above its answer.

1-5. Native Charm Architecture

1) What are the exact roles of `AlphaProVault`, `AlphaProVaultFactory`, `AlphaProPeriphery`, `ManagerStore`, and `IVault`?

`AlphaProVault` is the strategy vault (deposit/withdraw/rebalance and fee logic); `AlphaProVaultFactory` deploys clone vaults and controls protocol governance/fee state; `AlphaProPeriphery` is a read helper for positions/fees; `ManagerStore` is manager registration metadata; `IVault` is the interface contract surface integrators rely on.

2) What state is held in the vault versus inferred from Uniswap v3?

Vault contracts store policy and accounting state (shares, thresholds, fee settings, last rebalance metadata), while active liquidity composition, fee growth, and executable market conditions are inferred from Uniswap v3 pool state.

3) Which decisions are onchain and which are managerial/offchain?

Onchain enforces state transitions and permission checks; managerial/offchain decides rebalance timing, parameter updates, and operational runbooks. In 4626fun, additional offchain decisions exist in queue/keeper/CSW execution paths.

4) Is this strategy fully autonomous, semi-managed, or manager-dependent?

It is semi-managed but economically manager-dependent. Contracts are deterministic, but outcomes depend heavily on rebalance policy quality and operator liveness.

5) What assumptions are embedded in the three-order design (full-range, base, limit)?

The design assumes mean-reverting fee capture, manageable inventory drift, and timely rebalances. The one-sided residual leg assumes leftover directional inventory can be placed without becoming a systematic adverse-selection sink.

6-16. Deposit / Share / Withdrawal Mechanics

6) How are shares minted and burned?

Shares are minted/burned pro-rata against vault accounting state using Charm share math in native vaults, and ERC-4626 conversion logic in `Creator0Vault` for 4626fun wrappers.

7) What exactly does a share represent?

A share is a proportional claim on idle balances plus CLMM position inventory plus claimable fees, net of protocol/manager fee policy and subject to execution-path constraints.

8) On deposit, are user tokens immediately productive or can they sit idle?

They can sit idle. Native and wrapped flows can hold inventory until rebalance/deployment conditions are met.

9) Does share minting fairly account for idle balances, in-range/out-of-range liquidity, unpoked fees, and fee skims?

It is reasonably fair under normal cadence, but timing windows still exist around poke/rebalance and stale fee realization. Fairness is path-dependent, not absolute.

10) Does `getTotalAmounts()` represent a fair estimate of withdrawable value?

It is an accounting estimate, not a guaranteed synchronous withdrawal value.

11) How does `getPositionAmounts()` differ from actual exit value?

`getPositionAmounts()` is formulaic position accounting; it does not include execution slippage, market impact, or unwind path fragility.

12) Does fee accrual since the last poke create stale accounting or dilution risk?

Yes. Unpoked fee windows can shift value between cohorts and create temporary accounting drift.

13) Can tiny deposits, zero-burn pokes, or timing games change who captures fees?

Yes. Small timing-sensitive interactions can influence when fee state is realized and who benefits.

14) Are early depositors or withdrawers advantaged relative to later users?

Potentially yes, especially around stale accounting windows, rebalance boundaries, and stress exits.

15) Are there edge cases around first deposit / minimum liquidity / dust share issuance?

Yes. Native Charm has first-liquidity mechanics; 4626fun adds `_decimalsOffset`, minimum-first-deposit safeguards, and wrapper dust normalization (`userDustShares`) that must remain consistent.

16) Could users be misled if 4626fun wraps this with ERC-4626-style previews?

Yes. ERC-4626 previews can look synchronous while underlying exits remain path-dependent and sometimes queue-dependent.

17-25. Concentrated Liquidity Risk

17) How does the strategy allocate capital across full range, base range, and one-sided limit range?

Rebalance logic allocates into full/base ranges and then deploys residual inventory one-sided, making residual placement highly sensitive to direction and timing.

18) Under what market conditions is this design economically strong?

Mean-reverting, fee-rich, moderate-volatility regimes with disciplined rebalance cadence.

19) Under what market conditions does it become toxic?

Persistent trends, sharp gaps, thin-liquidity markets, and volatility spikes with informed flow.

20) How does it behave under trending, sharp gaps, chop, spikes, and prolonged one-sided flow?

Trending and gap regimes increase inventory damage; chop improves fee capture; spikes increase both fee opportunity and extraction risk; prolonged one-sided flow amplifies residual-order directional exposure.

21) Is the one-sided residual order a risk amplifier?

Yes. It can magnify directional inventory risk and informed-flow extraction.

22) How exposed is the vault to inventory imbalance, IL, adverse selection, JIT games, sniping, and toxic flow?

Materially exposed to all of these under weak guards or predictable rebalance timing.

23) Are there scenarios where fees look strong but inventory damage dominates?

Yes. Gross fee prints can mask net PnL deterioration from directional inventory drift.

24) Does the full-range slice reduce tail risk or mostly dilute returns?

It is a hedge-return tradeoff; in current 4626 deployment defaults (`fullRangeWeight=0`), that tail hedge is intentionally removed.

25) Could thresholds be too wide, too narrow, or structurally fragile?

Yes. Current deployment parameter choices are liveness-sensitive and can permit either over-churn or delayed adaptation.

26-38. Rebalance Logic and Guardrails

26) Walk through `rebalance()` line by line.

At high level: burn existing liquidity, collect/snapshot, compute new bands from current tick and thresholds, mint full/base/residual positions, then update rebalance metadata and fee activation state.

27) Are rebalance access controls safe?

Partially. They are conditionally strict and have a risk window before explicit delegate assignment.

28) What happens before `rebalanceDelegate` is set?

Rebalance can be externally callable, creating an adversarial timing surface.

29) Is "anyone can rebalance until delegate is set" acceptable or dangerous?

Situationally acceptable in a deliberate permissionless model, but dangerous in production without stronger guard envelopes and operational controls.

30) Are guardrails (`period`, `minTickMove`, `maxTwapDeviation`, boundary checks) strong enough?

They are explicit but parameter-dependent; poor calibration can either block needed action or allow economically bad action.

31) Can attackers manipulate price enough to pass/fail checks?

Near thresholds in low-depth conditions, yes.

32) Could manager/external callers force economically bad rebalances without violating rules?

Yes. Rule compliance does not imply economic optimality.

33) Does strategy create deterministic rebalance windows easy to front-run?

Yes, eligibility patterns can become predictable.

34) Can searchers extract around submission, range reset, and residual placement?

Yes. This is a core MEV surface.

35) Is TWAP protection sufficient against short-horizon manipulation?

Helpful but insufficient as a standalone anti-MEV defense.

36) Could stale TWAP or low-liquidity pool conditions make checks misleading?

Yes. Safety checks can be technically satisfied while economically misleading.

37) Are there states where vault should NOT rebalance even if code permits it?

Yes, especially fragile post-gap or thin-book states.

38) Are there states where vault SHOULD rebalance but code blocks it?

Yes, notably under overly strict deviation settings like `maxTwapDeviation=0`.

39-50. Valuation, Periphery, and Total-Assets Semantics

39) What is the purpose of `AlphaProPeriphery`?

Monitoring and analytics for positions/fees; not an execution-guarantee oracle.

40) Is periphery safe for monitoring only, or could teams misuse it for pricing/accounting?

Safe for monitoring, risky as sole pricing authority for mint/redeem fairness.

41) Does periphery understate or overstate true realizable value under stress?

It can do either; in practice it often appears optimistic because it omits real-time unwind costs and execution impact.

42) Are periphery fee calculations consistent with vault-side fee accounting?

Directionally yes under expected assumptions, but still not a substitute for executable-value estimation.

43) How much valuation drift can accumulate from unpoked fees, assumptions, out-of-range state, and stale spot?

Drift can become meaningful during volatile or delayed-maintenance periods and should be treated as a first-class risk metric.

44) If 4626fun presents a single NAV number, what is the right way to compute it?

Use a conservative hybrid model: accounting base, freshness checks, executable-liquidity context, and explicit haircut policy.

45) What is the wrong way to compute NAV?

Pure optimistic mark-to-market without freshness, liquidity, or unwind-path controls.

46) If wrapped in ERC-4626, what should `totalAssets()` mean in practice?

A conservative redeemable-equivalent estimate, not a best-case mark.

47) Should 4626fun use spot, pessimistic liquidation-style, TWAP, or hybrid valuation?

Hybrid with conservative bounds is safest.

48) Which valuation choice is safest for deposits?

Lower-bound/conservative valuation to prevent underpriced share minting.

49) Which valuation choice is safest for withdrawals?

Queue-aware executable estimate to avoid over-promising immediate value.

50) Where can 4626fun create silent insolvency or unfair-share bugs via valuation model errors?

At mint/redeem conversion boundaries when accounting value materially exceeds realizable exit value.

51-60. Governance and Privileged Control

51) Enumerate every privileged action the manager can perform.

Manager can set thresholds, periods, TWAP/deviation controls, weights, supply cap, pending manager fee, manager handoff, delegate, fee collection, emergency burn, and allowed sweep operations.

52) Enumerate every privileged action factory governance can perform.

Governance can update protocol fee, control governance handoff, and exercise protocol-level fee authority.

53) What can a rebalance delegate do?

Execute rebalances only; broader policy mutation remains manager/governance-controlled.

54) What can `emergencyBurn()` do economically?

Force liquidity removal/reset under stress, which can protect or harm depending on timing and market depth.

55) Could `sweep()` be abused or is it well-scoped?

It is relatively well-scoped when token0/token1 exclusions are enforced; still requires role hygiene.

56) Could threshold/fee/cap changes be used maliciously or irresponsibly?

Yes, materially. Economic harm can occur without violating code-level invariants.

57) Do fee changes activating only on rebalance create edge-case fairness issues?

Yes. Timing around flow windows can create discontinuous user outcomes.

58) Are there governance-latency/handoff risks with pending/accept flows?

Yes. Two-step handoffs reduce accidental mistakes but introduce transition risk windows.

59) Is manager operationally equivalent to an actively managed fund operator?

Yes, economically.

60) If yes, what disclosures are necessary for 4626fun users?

Disclose active-management dependence, mutable policy risk, accounting-vs-realizable divergence, queue/latency risk, and automation/oracle trust-boundary dependence.

61-64. Testing Adequacy

61) What does the provided test suite cover well?

Core happy paths, role gates, rebalance guards, valuation readiness checks, fallback paths, wrapper normalization, and ShareOFT validation behavior.

62) What major risks does the test suite miss?

Adversarial MEV conditions, stress exit fairness, prolonged automation outage behavior, and realistic market-impact unwind scenarios.

63) Are key stressed/adversarial categories missing?

Yes. Missing depth includes stressed exits, gap regimes, adverse selection, public rebalance abuse modeling, and full E2E outage/race testing.

64) Which 10 additional tests should be added first?

Rebalance sandwich edges, trend PnL decomposition, TWAP threshold manipulation, stale-poke mint fairness, wrapper queue-path parity, queue claim drift, persistent USDC unwind failure, dedupe race, owner-index drift recovery, and bundler/paymaster outage behavior.

65-72. 4626fun Integration Review

65) How exactly is Charm integrated into 4626fun?

4626fun deploys/uses native Charm vaults via `CreatorCharmStrategy`, then exposes a separate ERC-4626 layer (`CreatorOVault`) and a user wrapper (`CreatorOVaultWrapper` + ShareOFT).

66) What new assumptions are introduced by this integration stack?

Additional assumptions include oracle freshness for USDC conversion, queue liveness, wrapper normalization correctness, and CSW/automation execution reliability.

67) Where can ERC-4626 semantics diverge from concentrated-liquidity reality?

In previews, `totalAssets()`, and synchronous redemption expectations when unwind paths are constrained.

68) Could wrapper UX over-promise synchronous redemption?

Yes.

69) Could displayed TVL/share smoothness hide unstable inventory composition?

Yes.

70) Can stale strategy valuation create unfair mint/redeem outcomes?

Yes, and this is a high-severity integration risk.

71) Does batching reduce or increase fairness risk?

It can reduce timing games if valuation is robust, but increases unfairness if batch valuation is stale/optimistic.

72) How does async/queued behavior redistribute risk?

It shifts risk from immediate transactors toward queued users and remaining holders across delay windows.

73-80. Automation / CRE Review

73) Is rebalancing manual, script-driven, keeper-driven, or CRE-driven?

Hybrid today: direct workflow triggers, queue-based execution, and event-driven enqueueing coexist.

74) Which decisions should remain deterministic and onchain?

Auth checks, hard guard constraints, and accounting invariants.

75) Which decisions can be safely automated offchain?

Monitoring, queueing, validation, simulation, and deterministic relay/execution orchestration.

76) Which decisions should never rely on AI?

Authz, fee/threshold mutation, emergency writes, and any safety-critical execution gating.

77) If CRE is used, should it monitor/queue/validate/simulate/gate/execute?

Yes to monitor/queue/validate/simulate/gate; direct execution only as tightly constrained deterministic relay.

78) Does CRE improve observability/guardrails or just add complexity?

It improves observability and control surfaces, but current multi-path topology still adds avoidable complexity.

79) What extra failure modes does CRE/automation introduce?

Duplicate triggers, queue staleness, owner-index drift, bundler/paymaster outage, and cross-service API/DB coupling failures.

80) What is the minimal safe CRE role?

One canonical producer + one canonical executor with strict idempotency, replay protection, and deterministic policy checks.

81-98. Attack Scenarios

81) Public rebalance exploitation before delegate is set.

Attack path: external actor times public `rebalance` during adverse microstructure preconditions (`rebalanceDelegate=0`). Impact: unfair transfer/value drag (mostly permanent PnL loss, not just liveness). Detection: rebalance caller mismatch and post-rebalance slippage/perf anomalies. Mitigation: set delegate immediately, block pre-delegate rebalance, tighten guard envelope.

82) Manager-triggered bad rebalance.

Attack path: manager executes rule-compliant but economically poor rebalance. Impact: value loss from inventory repositioning. Detection: policy-change + rebalance telemetry divergence from baseline. Mitigation: timelock, bounded parameter changes, dual-control approvals.

83) Rebalance front-running/back-running.

Attack path: searchers position around predictable rebalance windows and range resets. Impact: MEV extraction and sustained strategy drag. Detection: abnormal before/after swap patterns and post-rebalance drift. Mitigation: private relay, timing jitter, stronger eligibility gating.

84) Deposit before rebalance to capture stale accounting.

Attack path: deposit into stale mark window before accounting catches up. Impact: cohort unfairness (value transfer). Detection: entry timing clusters around stale valuation windows. Mitigation: conservative mint valuation and freshness gating.

85) Withdraw after favorable poke and before unfavorable rebalance.

Attack path: user exits during temporary accounting advantage window. Impact: remaining holders absorb relative loss. Detection: clustered exits immediately after fee realization events. Mitigation: anti-timing controls and queue-aware policy.

86) Fee-capture timing game via poke updates.

Attack path: tiny transactions force state refresh and alter who captures fees. Impact: fairness distortion, usually medium severity. Detection: anomalous micro-deposit patterns around fee realization. Mitigation: deterministic

fee update cadence or batching policy.

87) Stale valuation in 4626 wrapper.

Attack path: stale oracle/conversion inputs feed optimistic mint/redeem math. Impact: unfair transfer and redemption expectation mismatch. Detection: stale-price telemetry with high flow. Mitigation: hard freshness gates and valuation haircuts.

88) Mispriced ERC-4626 share issuance.

Attack path: conversion uses optimistic accounting instead of executable value. Impact: silent fairness erosion, can resemble insolvency behavior in stress. Detection: widening gap between accounting NAV and realized exit outcomes. Mitigation: pessimistic executable valuation model.

89) One-sided residual order exploitation.

Attack path: informed flow trades against predictable residual side. Impact: directional inventory damage. Detection: repeated adverse fills near residual placement windows. Mitigation: adaptive residual sizing/placement policy.

90) Adverse selection during trend moves.

Attack path: persistent directional flow repeatedly selects against LP inventory. Impact: net value bleed despite fee income. Detection: trend-regime underperformance decomposition (fees vs inventory loss). Mitigation: faster adaptive rebalance + tighter trend controls.

91) TWAP guard bypass/insufficiency.

Attack path: near-threshold manipulation in low-depth windows. Impact: wrong rebalance decision pass/fail. Detection: boundary-edge triggers under thin-liquidity periods. Mitigation: longer windows, depth/liquidity-aware gating.

92) Forced rebalance near bad boundary conditions.

Attack path: trigger rebalance at technically valid but economically fragile boundaries. Impact: poor placement and follow-on losses. Detection: boundary-aligned rebalances with immediate adverse drift. Mitigation: add volatility/impact-aware veto conditions.

93) Manager fee change timing exploitation.

Attack path: schedule pending manager fee activation around expected flow windows. Impact: fairness/trust issue, medium severity. Detection: fee-change timing correlation with large user flows. Mitigation: notice windows, timelock, governance transparency.

94) Protocol fee change timing exploitation.

Attack path: governance times protocol fee changes around user activity. Impact: fairness discontinuity. Detection: governance-change and flow correlation monitoring. Mitigation: timelocked updates and explicit user notice.

95) `emergencyBurn` misuse or panic-action harm.

Attack path: emergency authority unwinds at poor market moment. Impact: direct value loss and liveness disruption. Detection: emergency-call audit logs and post-event PnL shock. Mitigation: restricted runbook, staged approvals, postmortem policy.

96) Offchain automation outage.

Attack path: keeper/queue/executor downtime stalls corrective operations. Impact: primarily liveness first, then economic drift/loss. Detection: missed SLO alerts, queue backlog growth. Mitigation: redundant executor path and tested failover runbook.

97) CRE double-trigger / duplicate action.

Attack path: overlapping producers/retries enqueue duplicates. Impact: churn, gas waste, potential extra MEV drag. Detection: duplicate dedupe-key collisions and repeated intents. Mitigation: single canonical producer and strict idempotent sink semantics.

98) Fake/optimistic NAV reporting to users.

Attack path: UI/API presents accounting as executable reality. Impact: trust damage and economically harmful user decisions. Detection: realized-exit outcomes diverge from displayed promises. Mitigation: always show accounting vs executable-now vs queued-estimate separately.

99-111. Stress Test Outcomes

99) 5% move within range.

Users usually observe small drift; accounting and exit values remain close; impact is mostly mark-to-market and typically auto-recovers with normal cadence.

100) 15% trend move through range.

Accounting lags inventory reality; exits degrade before policy catches up; early exits outperform late exits; recovery is manager/keeper-dependent.

101) 30% sharp move with thin liquidity.

Accounting can remain optimistic while real unwind is expensive/limited; user pain shifts to late redeemers; losses become partly realized; recovery requires active intervention.

102) Volatility spike with frequent whipsaw.

Users see unstable performance and rebalance churn; accounting oscillates; extraction risk rises; recovery depends on tuned policy and disciplined execution.

103) 50% TVL withdraw shortly after large move.

Buffer/exit bottlenecks dominate; many users face delay/revert pressure; fairness degrades strongly; recovery depends on queue mechanics plus operator actions.

104) Rebalance delayed for 6h / 24h / 72h.

Value divergence widens with delay length; user confidence and fairness decay; long delays can convert liveness risk into realized loss; recovery is operator-dependent.

105) Heavy fees accrue but no poke occurs.

Accounting under-realizes until refresh; users observe timing-sensitive fairness differences; usually recoverable once poke/rebalance occurs.

106) Manager changes thresholds aggressively mid-operation.

Users observe regime shift and unstable predictability; accounting assumptions break; losses can become realized if changes are poor; recovery requires governance rollback/retune.

107) Protocol/manager fee changes just before user flows.

Users experience discontinuous economics despite intact accounting math; fairness/trust impact is medium; recovery is policy/process, not automatic.

108) 4626 wrapper uses stale valuation.

Smooth displayed values hide execution mismatch; mint/redeem fairness can fail; affected cohorts bear cost; recovery requires strict freshness gates and repricing policy.

109) Automation/CRE does not fire.

Users observe stale state and delayed corrective actions; liveness fails first; economic damage accumulates over time; recovery is manual/runbook-driven.

110) Automation/CRE fires twice.

Users mostly see operational noise/churn; accounting often survives but costs/MEV drag increase; recovery needs idempotent action sinks and producer cleanup.

111) Searchers target predictable rebalance windows.

Users face persistent performance drag; accounting appears reasonable while realized outcomes degrade; costs are borne by LP holders; recovery requires private relay, timing jitter, and harder trigger policy.